

PART 23: FIVE END-TO-END PRODUCT JOURNEYS

23.0 What this section is and how to use it

The next five chapters take you through complete product builds — from initial idea to launched product to post-launch retrospective. Each is a master class disguised as a case study.

For each journey, you'll see:

- How the problem was identified.
- How discovery played out (what we asked, what we learned, what surprised us).
- The full PRD as it would actually be written.
- AI architecture decisions with explicit trade-offs.
- The metrics tree.
- The MVP plan with kill criteria.
- The trade-offs we made and why.
- The post-launch retrospective (real-feeling, including failures).
- A "What would Google look for here?" section that translates the build into APM-interview-relevant signals.

Read these slowly. The patterns recur across product types. Five different domains, but the underlying PM craft is the same.

The five journeys, with rationale for inclusion:

1. **Saarthi** — Consumer AI study companion for JEE/NEET aspirants. Tests: AI PM, consumer, Indian context, regulated-by-society category (education).
2. **ClearOps** — B2B SaaS ops platform for Indian D2C brands. Tests: B2B, integrations, multi-stakeholder dynamics.
3. **Mentora** — Tutor marketplace for Class 9-12 students. Tests: marketplace dynamics, cold start, trust.
4. **Vega** — Internal AI eval platform inside a hypothetical fintech. Tests: platform PM, internal customers, deprecation.
5. **Karya** — Autonomous email/calendar agent for knowledge workers. Tests: agentic AI, autonomy UX, the hardest of the five.

Total length: ~50,000 words across five journeys. Not because the brief asked for length but because each requires that depth to be complete.

JOURNEY 1: SAARTHI — AI Study Companion for JEE/NEET Aspirants

23.1.1 Origin and context

The idea for Saarthi emerged from a specific observation. India has approximately 1.5 million students taking JEE Main each year, another 2 million taking NEET. The vast majority pay ₹50,000-300,000 per year for coaching at offline institutes (Allen, Aakash, FIITJEE, Resonance, etc.). The marginal student in a Tier-2 or Tier-3 city often:

- Cannot access top-tier offline coaching.
- Studies from textbooks and YouTube videos with no personalized guidance.
- Has questions they can't ask anyone (parents are not domain experts; school teachers are overburdened).
- Practices on standardized test papers without diagnostic feedback.

Existing online platforms (Unacademy, Vedantu, PhysicsWallah, Byju's) provide content but not personalized coaching at scale. Their model is: produce great videos, sell access. The personal tutor experience is not their product.

The hypothesis: an AI tutor that knows the JEE/NEET curriculum deeply, adapts to each student's weaknesses, and provides Socratic dialogue could substitute for a fraction of the human tutor experience at 10x lower cost. If even directionally correct, the addressable market is large and growing.

The opening question: is this a real product, or just an obvious idea that's been tried 50 times?

23.1.2 Discovery — 40 student interviews

The first action was discovery, not building. Forty interviews over six weeks, structured as:

- 25 students currently preparing for JEE or NEET.
- 8 parents of students.
- 5 teachers at coaching institutes.
- 2 founders of competitor products who'd be willing to talk.

The interview structure (60 minutes each, Mom-Test-compliant):

Opening 15 minutes — context.

- "Walk me through your typical study week."
- "What does your average day look like Monday through Sunday?"

- "How do you decide what to study?"

Middle 30 minutes — pain.

- "Tell me about the last time you got stuck on a problem you couldn't solve."
- "What did you do?"
- "How did that feel?"
- "Tell me about your last mock test. How did it go? What was the worst part?"
- "What's the most frustrating thing about your prep right now?"

Last 15 minutes — workarounds and current solutions.

- "What tools or apps do you currently use?"
- "What do they do well? What do they do badly?"
- "If you could wave a wand and have any kind of help, what would it look like?"

Notes: never describe Saarthi or the proposed product. Listen for jobs, pains, and current solutions. Note specific phrases students used (these become product copy later).

What the interviews revealed

Real pain points (recurring across 30+ of 40 conversations):

1. **Lonely problem-solving.** "I get stuck for 30 minutes on one problem, then give up. I have nobody to ask. The coaching teacher is busy. My parents don't know physics." (Mentioned by 31 of 40.)
2. **No diagnostic feedback after mock tests.** "I get my score, but I don't know *why* I got those questions wrong. The institute doesn't go through them in detail." (28 of 40.)
3. **Topic prioritization confusion.** "There's so much syllabus. I don't know what to focus on. I just follow whatever order the institute uses." (24 of 40.)
4. **Concept gaps that compound.** "I realize 6 months later I never understood [Topic X], and now [Topic Y] is impossible." (22 of 40.)
5. **Anxiety and motivation.** "I have no one to talk to about how scared I am. My parents only ask about my marks. I feel alone." (19 of 40.)

Things students said they wanted that they probably don't really want:

- "Better video content" (they have unlimited content; the issue is *engagement* with content, not access).
- "More problems to solve" (they have unlimited problems; the issue is feedback on solutions).
- "Mock tests" (every institute provides these; the issue is post-test analysis).

This is the Mom Test in action. What students *say* they want vs. what they *actually need* differ. Discovery surfaces the latter.

Real surprises:

- A meaningful fraction of students described the social isolation explicitly. The product was initially conceived as a study tool; it became clear the product is also a companionship tool.
- Several students mentioned wanting an AI that could "judge them less than my parents do." Trust and non-judgmental dialogue surfaced as a feature requirement.
- WhatsApp as the primary expected delivery mechanism was almost universal. "Why would I download a new app? I live in WhatsApp."

Synthesis: the JTBD

After synthesizing 40 interviews, the JTBD became clear:

"When I'm stuck on a problem during my self-study or after a mock test, I want immediate, expert, patient guidance that helps me understand my weakness and shows me a path forward — without making me feel stupid or alone — so I can keep studying productively and trust I'm progressing."

This is the job. Notice what's missing: there's nothing about content. Students don't lack content. They lack a tutor.

The product is fundamentally not "more content." It's a tutor.

23.1.3 Concept testing — 12 students with mockups

Before building anything, I built clickable Figma mockups of the core flow:

1. Student opens WhatsApp, messages the bot.
2. Bot asks: "What are you stuck on?" Student types or photographs the problem.
3. Bot responds with a Socratic dialogue: not the answer, but the next step in reasoning.
4. Student responds; bot adapts.
5. After resolving, bot logs the interaction; over time, a profile forms of the student's strengths and weaknesses.
6. Periodic check-ins: "I notice you've struggled with rotational mechanics three times this week. Want to walk through it from scratch?"
7. Mock-test review mode: paste your answers; bot reviews each, identifies pattern.

Showed mockups to 12 of the original 40 students. Asked: "If this existed today, would you use it?"

Reactions:

- 10 of 12 said yes, with varying enthusiasm.
- 2 said "maybe" — both said they'd want to see it actually work, not just mockups.

- One specific concern: "Will it actually be helpful or will it just give me wrong answers?"

This last concern became the central product risk: AI hallucinations on technical content. If Saarthi confidently teaches a wrong concept, it's worse than nothing. The eval discipline (covered later) is non-negotiable.

23.1.4 The PRD

Below is the PRD as it would be written before building the MVP. Real-feeling, with all 15 sections.

PRODUCT REQUIREMENTS DOCUMENT

PRODUCT: Saarthi — AI Study Companion for JEE/NEET Aspirants

VERSION: v0.1 (MVP)

AUTHOR: [PM name]

DATE: April 2026

STATUS: Draft for review

REVIEWERS: Eng lead, AI lead, Designer, Founder

1. ONE-LINER

Saarthi is a WhatsApp-based AI tutor for Indian JEE/NEET aspirants that helps students work through problems they're stuck on, identifies their weak topics over time, and provides personalized study guidance – at ₹500/month vs. ₹5,000+/month for offline coaching.

2. CONTEXT

The Indian JEE/NEET prep market has ~3.5M annual aspirants and ~₹50K-3L ARPU at offline coaching institutes. The Tier-2/3 segment (estimated 60%+ of aspirants) is underserved: limited access to top coaching, parents who can't help with technical content, and anxiety from isolation in study.

Online incumbents (Unacademy, Vedantu, PhysicsWallah, Byju's) compete on content production. None offer personal tutoring at scale. Frontier LLMs (GPT-4 class, Claude Opus) can now reason through JEE/NEET-level problems with sufficient grounding, opening the technical possibility of AI-personalized tutoring.

Discovery (40 student interviews) confirmed:

- The pain is acute (~78% of interviews mentioned getting stuck without help as a top frustration).
- The current workarounds (YouTube, asking peers, abandoning the problem) are unsatisfying.
- Trust is high if the product proves accurate; trust collapses immediately on factual errors.

The window: this market segment is digitally native, comfortable with WhatsApp, and price-sensitive – perfect for a low-cost AI-first product.

3. PROBLEM STATEMENT

Indian JEE/NEET aspirants in Tier-2/3 cities, ages 16-19, lack access to personal tutoring during their 1-2 years of preparation. They face:

- 31 of 40 interviewees cited "getting stuck on problems with no one to ask" as a top pain.
- 28 of 40 cited absence of post-mock-test diagnostic feedback.
- 24 of 40 cited topic prioritization confusion.
- 22 of 40 cited cumulative concept gaps that compound over months.
- 19 of 40 mentioned anxiety/isolation related to study.

The cost: students underperform their potential, exit JEE/NEET prep without satisfactory results despite parental investment, and develop negative associations with rigorous study.

4. USERS AND USE CASES

Primary persona: "Tier-2 Aspirant"

- Ages 16-19, mostly male (60/40 male/female reflecting JEE/NEET participation), Tier-2/3 cities (e.g., Patna, Indore, Coimbatore, Lucknow).
- Student in Class 11 or 12, or Drop Year (gap year focused on preparation).
- Family income: ₹4-15 lakh per year (middle class).
- Smartphone: budget Android (Xiaomi, Realme, Vivo); 4G mostly; data cost-conscious.
- Already pays ₹50,000-150,000/year for some form of coaching (institute or online platform).
- Uses WhatsApp 4+ hours/day. YouTube. Some Telegram for study groups.
- English-comfortable but not native; Hindi/regional often preferred for casual conversation.

Secondary persona: "Parent of Aspirant"

- Often the buyer (pays for the product); not the user.
- Wants visibility into child's progress without micromanaging.
- Skeptical of AI; needs reassurance about accuracy.

Top 5 use cases (priority order):

UC1: Stuck-on-problem help.

Student is solving a problem from a textbook or mock test. Stuck for >10 minutes. Opens Saarthi on WhatsApp. Photo or types the problem. Saarthi guides them through the reasoning step-by-step.

UC2: Mock-test review.

Student finishes a mock test. Pastes/photos their answers and the question paper. Saarthi identifies which questions are wrong, why, and groups by weak topics.

UC3: Topic learning.

Student wants to learn or revise a topic. "Explain rotational dynamics to me." Saarthi provides explanation, examples, and practice problems adapted to the student's level.

UC4: Daily study planning.

"What should I study today?" Saarthi reviews recent struggles, recommends 60-90 minutes of focused work targeting weak areas.

UC5: Motivational/anxiety check-ins.

Saarthi periodically (1-2x per week) checks in: "How are you feeling about your prep?" Provides empathetic listening and grounded advice.

5. PROPOSED SOLUTION

Saarthi is delivered via WhatsApp. Three primary modes:

A. Reactive mode (student-initiated): Student messages Saarthi with a problem, photo, or question. Saarthi responds with Socratic guidance.

B. Proactive mode (Saarthi-initiated): Daily check-ins, weekly progress summaries, periodic anxiety check-ins.

C. Review mode (mock-test): Student pastes their attempt; Saarthi diagnoses each question.

Architecture:

- WhatsApp Business API (via Twilio or AiSensy) for message I/O.
- Backend: Python service with FastAPI.
- LLM layer: GPT-4o or Claude Sonnet for reasoning; smaller model (GPT-4o-mini or Claude Haiku) for routing and simple queries.
- RAG layer: indexed corpus of NCERT textbooks, JEE/NEET past papers, and curated explanations from public domain sources.
- Eval system: 500-question gold dataset; LLM-judge pipeline; weekly regression testing.
- User profile DB: Postgres tracking topic-level mastery scores per student.

Default flow for "stuck on a problem":

Student: [photo of physics problem]

Saarthi: I see you're working on a problem about [topic detected from OCR + analysis]. Before I help, let me check – what have you tried so far? What confused you?

Student: I drew a free-body diagram but I don't know what to do next.

Saarthi: Good – a free-body diagram is the right start. Let's look at it together. [If photo of diagram, OCRs it.] Can you list the forces you've drawn? I want to make sure we have all of them.

Student: Gravity, normal force, friction.

Saarthi: Great. Are there any other forces I should consider? Hint: this object is on an inclined surface and being pushed.

[continues until student arrives at solution OR Saarthi escalates with more direct help]

Note: Saarthi does NOT give the answer immediately. Socratic dialogue is the design choice. Direct-answer mode is available but secondary.

=====

6. SUCCESS METRICS

Primary metric: Weekly Active Users (WAU). Target: 1,000 WAU by month 3 of MVP launch.

Secondary metrics (guardrails):

- Resolution rate (% of student questions ending with student affirming understanding): target 70%+.
- Retention (Week 4 retention from sign-up): target 50%+.
- Hallucination rate (manual sample review): target <2% of responses.
- Conversion (free trial → paid subscriber): target 15%+ at month 6.

Negative metrics to watch:

- Per-user inference cost: target <₹400/month per active user.
- User-reported errors (in-app "report" button): keep below 5% of sessions.
- Parent satisfaction: NPS >40.

Time horizon: monthly checkpoints; first major review at 90 days post-launch.

=====

7. NON-GOALS

Saarthi is NOT:

- A content library. We will not produce video content. (Many competitors do. We don't.)
- A test-taking platform. We will not host mock tests. We will help students review tests they took elsewhere.
- An offline app. Internet-required. (We may add limited offline capability later, but not in MVP.)
- A gamified study tracker. No streaks, no leaderboards, no badges. (Gamification is engagement-bait that doesn't serve actual learning.)
- A school replacement. Saarthi supplements coaching/study; does not replace.
- For students below Class 9 or above first-year college. Out of scope.
- A general-purpose AI chatbot. Saarthi only discusses JEE/NEET prep; off-topic queries are politely deflected.

8. EDGE CASES

E1: Student photo is unreadable. Behavior: Saarthi asks student to retake or type the question. Provides a sample of how to type math notation.

E2: Student question is in Hindi or regional language. Behavior: Saarthi accepts; responds in same language. (Initial scope: Hindi + English. Tamil, Telugu, Bengali in v1.1.)

E3: Student asks an off-topic question (e.g., "What's a good movie?"). Behavior: Saarthi politely deflects. "I'm here to help with your JEE/NEET prep. Want to work on a topic?"

E4: Student asks something safety-relevant (mental health, suicidal ideation). Behavior: Saarthi expresses care, provides Vandrevalla or iCall hotline numbers, encourages talking to trusted adult. No attempt to handle as therapy.

E5: Student is wrong but confident. Behavior: Saarthi gently corrects with explanation and verification. "Let me show you why – try this specific case."

E6: Student tries to get Saarthi to do their homework wholesale. Behavior: Saarthi explains its role: "I help you learn, not do your work. Let's take it step by step."

E7: Saarthi doesn't know the answer with confidence. Behavior: Saarthi explicitly says "I'm not 100% sure on this – let me try a different approach" or "Please verify this with your teacher." Does not pretend to know.

E8: Student in poor connectivity. WhatsApp messages may delay. Behavior: messages queue and deliver when connection resumes; Saarthi is patient with delays.

E9: Adversarial use – student tries to get Saarthi to misbehave (jailbreak prompts). Behavior: system prompt is hardened; refuses clearly and re-anchors to JEE/NEET prep.

E10: Multi-student account sharing. Behavior: detected via usage pattern; product requires student-level signup; family plans available for siblings.

9. RISKS AND MITIGATIONS

R1: Hallucinations on technical content (HIGH likelihood, HIGH impact).
Mitigation: Eval system with 500-question gold dataset; monthly regression testing; in-product "report error" button; manual sampling of 1% of conversations weekly; LLM-as-judge pipeline.

R2: Insufficient retention (MEDIUM likelihood, HIGH impact).
Mitigation: Track Week 1, Week 4, Week 12 retention separately. If Week 4 < 30%, treat as kill criterion. Iterate on engagement loops.

R3: Cost per user > ₹400/month at scale (MEDIUM likelihood, HIGH impact). Mitigation: Model cascade (cheaper model for simple queries, expensive only for hard reasoning); aggressive caching of similar explanations; Indian Open-weight models (e.g., Sarvam) for routing where quality permits.

R4: Negative parent reaction to AI (MEDIUM likelihood, MEDIUM impact).
Mitigation: Parent-facing comms emphasizing safety and accuracy; weekly progress reports; explicit acknowledgment of AI limitations; human support channel.

R5: Regulatory uncertainty (LOW likelihood, MEDIUM impact). The edtech space in India has DPDP Act implications and potential CBSE/MOE oversight. Mitigation: Privacy by Design from day one; legal review of comms; no claims of "guaranteed results."

R6: Competitor response (Byju's, Unacademy launch competing AI). (MEDIUM likelihood, MEDIUM impact). Mitigation: speed and depth of focus on JEE/NEET (vs broad K-12); brand and community.

=====

10. DEPENDENCIES

D1: WhatsApp Business API access (via partner like AiSensy). Lead time: 2-3 weeks for Meta approval. Owner: Engineering.

D2: NCERT textbook content (publicly available; needs cleanup and indexing). Lead time: 4 weeks for full corpus. Owner: AI team.

D3: JEE/NEET past paper corpus (public; needs OCR and structured parsing). Lead time: 6 weeks. Owner: AI team.

D4: Eval gold dataset (500 questions with expert-validated solutions). Lead time: 4 weeks. Need to hire 2 expert reviewers (subject matter experts in physics, chemistry, biology, math). Owner: PM + AI team.

D5: Payment integration (Razorpay for ₹500/month subscription). Lead time: 1 week. Owner: Engineering.

=====

11. ROLLOUT PLAN

Phase 1 (Weeks 1-4 post-build): Closed alpha with 30 students recruited from original interview pool. Free access. Heavy feedback collection.

Phase 2 (Weeks 5-8): Open beta with 200 students. Free 30-day trial. Monitor metrics; iterate.

Phase 3 (Weeks 9-12): Public launch. ₹500/month subscription with 14-day free trial. Marketing through Instagram + JEE/NEET YouTube creators + WhatsApp study groups.

Phase 4 (Months 4-6): Geographic expansion (start in Bihar/UP/Tamil

Nadu Tier-2 cities; expand based on demand).

12. EXPERIMENTS / VALIDATION

Pre-launch:

- Concierge MVP test (Weeks 1-2): I personally respond to 5 students' questions over WhatsApp, handcrafted, mimicking what Saarthi would do. Validate the format works.
- LLM benchmark (Week 1-2): Test GPT-4o, Claude Sonnet, and open-weight Sarvam on 100 sample JEE/NEET problems. Pick the model with best correctness × cost ratio.

Post-launch (in beta):

- Socratic vs direct-answer A/B test: half of beta users get default Socratic; half get default direct-answer. Measure satisfaction, retention, learning outcome (self-reported and via mock test performance).
- Pricing test (Phase 3): some users see ₹300, some see ₹500, some see ₹700/month. Measure conversion.

13. OPEN QUESTIONS

Q1: Should we offer a parent-facing dashboard? Pro: parental visibility might increase willingness to pay. Con: students may feel surveilled, reducing trust. Decision pending after Phase 1.

Q2: Should free tier exist? Currently scoping 14-day free trial only. Consider freemium (limited messages/day) if conversion is too low.

Q3: How do we handle academically dishonest use (Saarthi solving homework wholesale)? Initial behavior is to refuse; may need to revisit based on student behavior data.

Q4: What's our long-term position vs. content platforms (Unacademy, PhysicsWallah)? Are we their replacement or complement?

14. APPENDICES

- A. Discovery interview notes (40 interviews, anonymized).
- B. JTBD synthesis.
- C. Concept test results (12 students).
- D. Competitor analysis (10 products).
- E. AI architecture detailed design.
- F. Eval framework design.
- G. Privacy review checklist.

15. SIGN-OFFS

- ☐ Engineering lead
- ☐ AI lead
- ☐ Design lead
- ☐ Founder
- ☐ Legal/compliance

This PRD represents about 8-10 hours of dedicated work. Most PRDs at most companies are weaker. This one is real-feeling: it has specifics, it makes commitments, it acknowledges uncertainty, it explicitly lists what we won't do.

23.1.5 AI architecture decisions

The AI architecture is where the product lives or dies. Here are the key decisions, with explicit trade-offs.

Decision 1: Frontier model vs open-weight model

Choice: Frontier model (GPT-4o or Claude Sonnet) for primary reasoning.

Reasoning: JEE/NEET physics, chemistry, biology, and math problems require strong reasoning. Open-weight models (Llama 3.1, Mistral, Sarvam) at the time of writing are still ~10-20% behind frontier models on technical reasoning. The cost difference (₹50/M tokens vs ₹500/M tokens) is significant but the quality gap matters more for our use case — wrong physics is worse than expensive physics.

Trade-off accepted: Higher per-query cost (~₹2-4 per student question vs ~₹0.30 for open-weight). This pushes our unit economics: at ₹500/month subscription and ~30 questions/student/day, COGS is ~₹120/month/student. Margin is workable but not lavish.

Mitigation: Cascade architecture. Use a smaller/cheaper model for routing decisions (which subject, what type of question, easy/hard). Only invoke the frontier model for the actual reasoning. This cuts cost ~40%.

Decision 2: RAG vs fine-tuning vs in-context

Choice: RAG primary; in-context for few-shot examples; no fine-tuning in v1.

Reasoning:

- The JEE/NEET corpus (NCERT textbooks, past papers, key reference materials) is large and specific. RAG is the right tool to ground responses.
- Fine-tuning would require curated training data we don't have at v1 scale. Also: fine-tuning a frontier model is expensive and we lose model upgrades automatically.
- Few-shot examples in the prompt help with format and tone.

Trade-off accepted: RAG quality depends entirely on retrieval quality. Bad retrieval = bad response. We invest heavily in:

- Chunk strategy (per-section for textbooks; per-question for past papers).
- Hybrid search (semantic + keyword) because exact terms matter ("rotational kinematics" should retrieve rotation-specific content).
- Re-ranking (use a cross-encoder to re-rank top-K results).

Decision 3: Eval framework

Choice: 500-question gold dataset, expanded over time; LLM-as-judge for routine eval; human review for 1% sample weekly; regression testing on every prompt change.

Why this matters more than the model choice: The model is interchangeable; the eval set is what tells us if we improved. We invest 4 weeks upfront to build the gold dataset (paying expert tutors to validate), and continue expanding.

The eval categories:

- Accuracy: Is the final answer correct?
- Reasoning quality: Are the steps sound?
- Pedagogical quality: Does it teach without giving answers prematurely?
- Tone: Is it patient, non-judgmental?
- Safety: Does it refuse to handle adversarial prompts cleanly?

For each category, score 1-5 by LLM-judge (calibrated against human review samples).

Decision 4: Hallucination defense layers

The most important problem in this product. Layered defenses:

1. **Grounding via RAG.** Most factual content comes from indexed corpus; reduces hallucination.
2. **Citation.** When Saarthi references a textbook fact, it cites: "From NCERT Class 12 Physics, Chapter 5..."
3. **Confidence prompting.** System prompt explicitly tells the model to say "I'm not sure" when uncertain.
4. **Verification step.** For numerical answers, a second model pass verifies the calculation.
5. **User reporting.** In-app button to flag errors. Each flag triggers human review and (if confirmed) addition to a "regression test" suite.
6. **Manual sampling.** 1% of conversations are reviewed weekly by a human expert.
7. **Topic boundaries.** Saarthi explicitly does not engage with questions outside the JEE/NEET syllabus.

These are layers because no single defense is sufficient. Defense in depth.

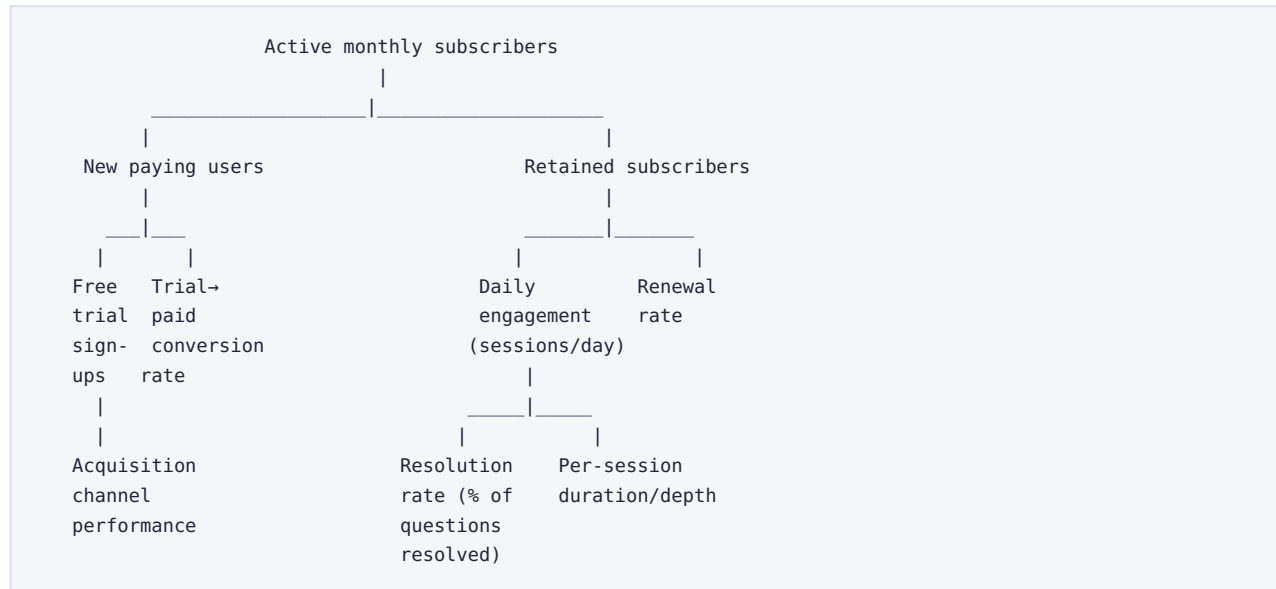
Decision 5: Latency target

Choice: Median response time <8 seconds; P95 <15 seconds.

Why these numbers: For a chat experience, anything over 10 seconds feels broken. WhatsApp's expected cadence is fast. But: deep reasoning can't be rushed without quality cost. Streaming partial responses (showing "thinking..." then incremental output) helps perceived latency.

Trade-off accepted: Some queries are slow because reasoning is slow. We optimize what we can (model choice, prompt length, parallel retrieval) and accept the rest.

23.1.6 The metrics tree



The North-Star Metric: **Active monthly subscribers** (proxy for value delivered + business sustainability).

The leading indicator we watch most carefully: **Resolution rate × Per-session duration**. If users are getting their problems resolved and engaging deeply, retention and conversion follow.

The early warning indicator: **Hallucination rate × User error reports**. If these climb, retention is about to crater regardless of what other metrics show.

23.1.7 The MVP plan with kill criteria

MVP Scope (4 months to launch):

Month 1: Core architecture

- WhatsApp integration.
- LLM cascade.
- Basic RAG over NCERT.
- Initial system prompts.
- 100-question eval set.

Month 2: Core flow

- Stuck-on-problem flow.
- OCR for problem photos.

- User profiling and topic mastery tracking.
- Concierge MVP with 5 students.

Month 3: Polish + closed alpha

- Mock-test review flow.
- Daily check-ins.
- Eval set expanded to 500.
- 30 students in closed alpha.
- Internal hardening based on alpha feedback.

Month 4: Open beta + payment

- Public launch in 1-2 cities.
- Razorpay integration.
- Pricing test.
- 200 users target by end of month.

Kill Criteria (decide at month 3 of public launch, ~month 7 from start):

- If Week 4 retention <30%: kill. The product isn't fundamentally landing.
- If hallucination rate >5%: kill or radical re-architecture.
- If per-user cost > ₹600/month against ₹500 ARPU: kill or reprice.
- If conversion (trial to paid) <8%: kill or fundamentally rethink value prop.

If any of those triggers, we stop, learn, and either pivot or shut down. Pivots considered: B2B (selling to coaching institutes); narrower scope (just mock-test review); broader scope (general K-12 tutor).

23.1.8 Trade-offs we accepted

Eight specific trade-offs, named explicitly:

1. **Quality > Speed.** Used frontier model despite cost. Will hurt margin until scale.
2. **Socratic > Direct.** Default to Socratic dialogue, even though students sometimes prefer the direct answer. We're betting on learning outcomes over instant gratification.
3. **WhatsApp > App.** Lower friction on adoption; less control over UX; stuck with WhatsApp's constraints.
4. **JEE/NEET only > broader.** Focused scope; missing addressable market in K-12 broadly. Bet that depth in JEE/NEET wins.
5. **Subscription > freemium.** Risk lower acquisition; better unit economics. Will revisit if conversion is low.
6. **Eval-first > ship-fast.** 4 weeks on gold dataset before MVP. Slower start; better quality.

7. **No video content.** Limits what we can offer relative to competitors. Bet that tutoring is differentiated enough to win.

8. **No gamification.** Possibly lower engagement; bet that real learning > engagement-bait.

Each is a real trade-off. Each has costs we accept.

23.1.9 Post-launch retrospective (hypothetical, real-feeling)

After 9 months in market:

What worked:

- Stuck-on-problem flow was the killer feature. 78% of student WAU touched this flow weekly.
- Mock-test review was second strongest. Higher session depth than expected.
- WhatsApp distribution was correct. Friction-free adoption.
- Hallucination rate stayed under 1.5% — better than target. Eval discipline paid off.
- Word-of-mouth in study groups was a real acquisition channel.

What didn't:

- Daily check-ins were almost universally ignored. Removed in v1.2.
- Anxiety/motivation flow was used by maybe 5% of users; dropped as a default.
- Parent dashboard launched in v1.1; very low usage. Repackaged as a weekly digest email instead.
- Conversion (trial → paid) settled at 11% — below our 15% target. Forced us to extend trial to 30 days and repackage entry-tier.
- Retention W4 was 38%, below 50% target. Investigation showed: students who used Saarthi 3+ times in first week retained at 60%+; students who used it 1-2 times retained at 15%. Conclusion: onboarding push needed to drive deeper first-week engagement.

What we changed:

- Onboarding now includes a guided first-problem walkthrough (pre-set sample problem walked through Socratically). First-week engagement up 30%.
- Pricing repackaged: ₹400/month basic (with monthly question cap), ₹800/month unlimited. Conversion improved.
- Removed daily check-ins; added "weekly progress" digest instead. Less intrusive, similar visibility for parents.
- Geographic expansion: started Hindi-language UI in v1.1. Added Tamil and Telugu in v1.3.

What we'd do differently from scratch:

- Move to onboarding-driven engagement push earlier. We assumed students would use the product naturally; many didn't.
- Less ambitious feature scope at MVP. Daily check-ins and motivation flows distracted from the core "tutor" use case.
- Earlier WhatsApp Group integration. Many students study in groups; we eventually built a "study group" mode in v1.4 that drove retention.
- More aggressive eval expansion. We launched with 500 questions; should have launched with 1500.

What's next:

- Voice input/output for students who type slowly.
- Parent-facing companion (separate WhatsApp number) with summaries and concerns.
- Tier-3 city expansion with vernacular-first onboarding.
- Long-term: school/coaching partnerships for B2B distribution.

23.1.10 What would Google look for here?

If you were a Google APM candidate presenting Saarthi as your product story, what signals does this case study send?

Strong signals:

- Discovery rigor (40 interviews, JTBD synthesis, concept testing before building).
- AI fluency (clear cost/quality/latency thinking, eval-driven development, layered hallucination defense).
- Trade-off explicitness (8 named trade-offs with costs accepted).
- Metric thinking (NSM tree, leading indicators, kill criteria).
- Indian-context awareness (WhatsApp distribution, Tier-2/3 focus, vernacular).
- Honest retrospective (named what didn't work, not just what did).

What would prompt follow-up questions:

- How did you decide on the ₹500 price point specifically? (Test of strategic pricing reasoning.)
- What if a competitor with deeper pockets (e.g., Byju's) launches a similar AI product 6 months later? (Test of competitive strategic thinking.)
- The retention number is below target. Beyond what you mentioned, what other hypotheses could explain it? (Test of analytical rigor.)
- Why Socratic over direct? Walk me through the trade-off more deeply. (Test of educational philosophy + product taste.)

- What's your view on the long-term defensibility of an AI-tutor product? (Test of strategic thinking on AI moats.)

A strong candidate handles these follow-ups with specific reasoning, not generic answers. The Saarthi case study, fully internalized, prepares you for all of these.

This is what an end-to-end product journey looks like when documented. Notice what's not here: theatrics, big numbers, magical insights. Just discovery → decisions → outcomes → learning.

The next four journeys follow the same pattern in different domains. Each should expand your range. Read all five.

End of Journey 1. Journey 2 — ClearOps — begins next.

\newpage

JOURNEY 2: CLEAROPS — AI-Powered Ops Platform for Indian D2C Brands

23.2.1 Origin and context

Indian D2C — direct-to-consumer — brands have exploded since 2018. Boat (audio), Mamaearth (personal care), Sugar (cosmetics), HealthKart, BlueSmart, dozens more. The pattern: build a brand, sell through Amazon + Flipkart + own website + Myntra + offline retail.

The operational complexity is severe. A single D2C brand selling on 5+ marketplaces faces:

- Inventory split across multiple warehouses (their own + Amazon FBA + Flipkart + others).
- Marketplace-specific listing requirements, pricing rules, and promotion policies.
- Returns flowing back through different reverse-logistics paths.
- Reconciliation between marketplace payouts (which lag, get adjusted, get disputed).
- Demand forecasting across channels with different velocity profiles.
- Customer support across channels (Amazon's customer chat, Flipkart's, their own, Instagram DMs).

Most D2C brands run this with 3-7 ops people in spreadsheets, shared Google Sheets, and a hodgepodge of marketplace dashboards. The cost is significant — typically 8-15% of operations effort goes into reconciliation and coordination work that should be automated.

The hypothesis: an AI-powered ops platform that integrates with all major marketplaces, gives a unified view, automates reconciliation, and provides predictive demand insights would save Indian D2C brands meaningful money and time. Ramp/Notion-quality UX in this category doesn't exist for Indian D2C-specific needs.

23.2.2 Discovery — 25 interviews

Twenty-five interviews:

- 15 ops leads or founders at Indian D2C brands (₹5-50 Cr ARR).
- 6 customer support managers.
- 4 ops/finance people at the larger D2C brands (Boat-scale).

What we asked:

- "Walk me through your ops day."
- "When did inventory or reconciliation last go wrong? What happened?"
- "What's your worst monthly ritual?"
- "Which dashboard do you check most often? Why?"

What we learned:

Pain 1: Reconciliation hell. Every D2C brand had a horror story about marketplace reconciliation. Amazon and Flipkart payouts come 14-30 days after sale, with myriad deductions (FBA fees, marketing, returns, commissions, platform fees, GST). Matching payout to actual orders takes 8-20 hours per month per marketplace. Errors cost real money — typically ₹1-5L per month leaks because nobody catches discrepancies.

Pain 2: Inventory blindness. "I have 3,000 units across 5 channels. Where are they? Which are in transit? Which are stuck? Which warehouses need rebalancing?" Nobody could answer in real time. Spreadsheets lag 1-3 days. Manual.

Pain 3: Demand forecasting. "I order from manufacturer 60-90 days in advance. I have to guess. If I'm wrong, I either stock out (lose sales) or overstock (cash trapped + clearance discounts)."

Pain 4: Cross-channel coordination. Promo on Amazon doesn't update Flipkart pricing. Stockout on Myntra doesn't trigger reorder. "I have to remember everything."

Pain 5: Customer support fragmentation. Customer messages from Amazon, Flipkart, Instagram, WhatsApp, own website. Different teams, different tools, no unified history per customer.

The killer insight: every brand we talked to had built a janky internal "ops dashboard" in Google Sheets + Zapier. They'd already paid the price of the missing tool. They were waiting for a real one.

JTBD:

"When I'm running operations for my D2C brand across multiple marketplaces, I want unified visibility, automated reconciliation, and predictive insights — so I can stop spending half my week on coordination and reconciliation, and focus on growing the brand."

23.2.3 The PRD (compressed)

Product: ClearOps — Operations platform for Indian D2C brands selling across Amazon, Flipkart, Myntra, own website, Meesho, and offline.

Core modules:

1. **Unified inventory dashboard.** Real-time view of stock across channels and warehouses.
2. **Reconciliation engine.** Auto-matches marketplace payouts to orders; flags discrepancies for review.
3. **Demand forecasting.** ML-driven forecasts per SKU per channel, 30/60/90 day horizons.
4. **Cross-channel pricing and promo orchestration.** Push price/promo changes across marketplaces.
5. **Unified customer view.** Single timeline per customer across all channels.

Primary users: Ops manager at a D2C brand, ₹3-100Cr ARR, currently using spreadsheets.

Pricing: Tiered SaaS — ₹15K/mo (small, up to ₹5Cr ARR), ₹40K/mo (mid, ₹5-30Cr ARR), ₹1L+/mo (enterprise, ₹30Cr+).

Success metrics:

- Active brands: 200 by month 12.
- Net Revenue Retention: 120%+ (expansion within accounts).
- Reconciliation time saved per customer: 70%+ vs spreadsheet baseline.
- Forecast accuracy: MAPE < 25% per SKU at 60-day horizon.

Non-goals:

- We're not a marketplace integrator (Shiprocket, Unicommerce). We're orchestration on top of those.
- We're not building manufacturing/supply chain modules.
- We're not a customer support platform (Freshdesk, Zoho Desk own that). We provide the unified view; tickets go elsewhere.

23.2.4 AI architecture decisions

This is a B2B product where AI is one of several differentiators, not the entire product. The architecture is hybrid: traditional SaaS infrastructure for data integration and dashboards; AI for the genuinely AI-suited tasks.

Where AI sits:

1. **Reconciliation logic.** The marketplace payout files have inconsistent formats, line items, and naming. An LLM with structured output is excellent at parsing varied PDFs and CSVs into a clean reconciliation format. We use a smaller LLM (Claude Haiku or GPT-4o-mini) for parsing; rules engines for matching.
2. **Demand forecasting.** Classical ML, not LLM. Time-series models (Prophet, ARIMA-X, gradient boosting) with features for seasonality, promotion, marketplace velocity. Forecasting is a deterministic problem; LLMs are wrong tool.
3. **Anomaly detection.** Identify weird patterns in sales data — sudden drops, unusual returns, marketplace deductions that don't match historical norms. Combination of statistical methods + LLM-based reasoning for the "what does this anomaly mean" part.
4. **Customer support summarization.** When ops opens a customer's history, AI generates a 2-sentence summary of recent interactions across channels.
5. **Natural language query.** "Show me Amazon Bangalore inventory for SKUs X, Y, Z." LLM translates to SQL/data queries against our internal data warehouse. This is increasingly table stakes for B2B SaaS in 2026.

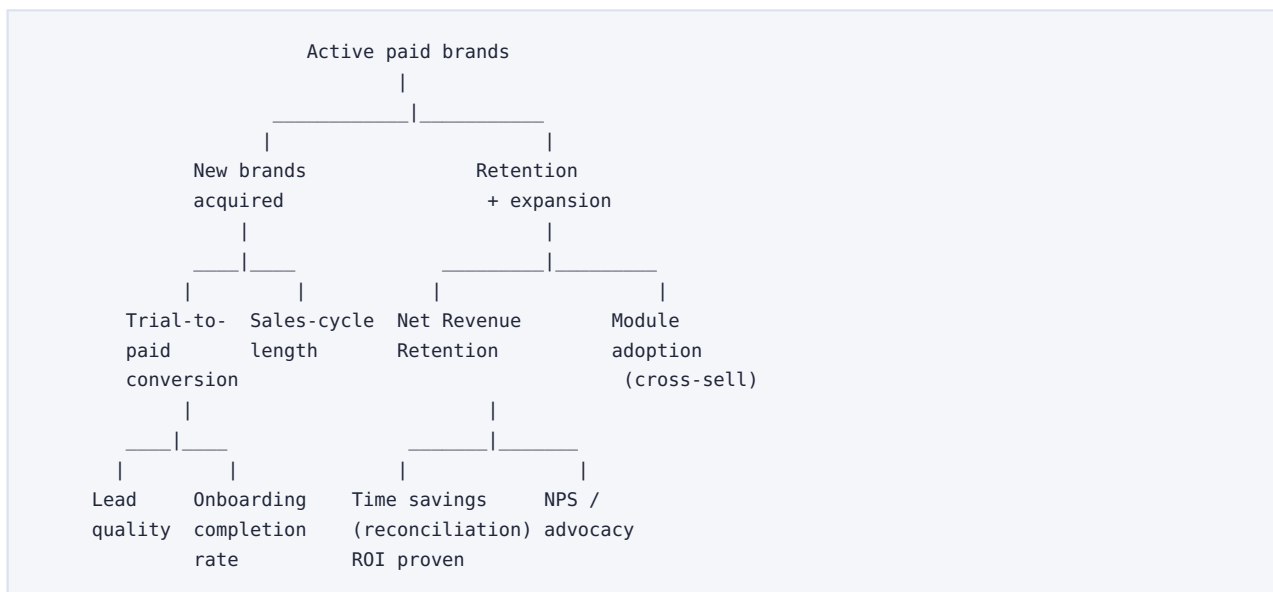
Where AI does not sit:

- Order routing: rules-based (deterministic, auditable).
- Inventory movements: rules-based (auditable).
- Pricing/promo orchestration: rules-based with human approval for cross-channel pushes.

Why this discipline matters: B2B customers do not tolerate non-determinism in operations. Inventory cannot be "probably moved." Reconciliation cannot have hallucinations. AI is for cognition; rules are for state.

This is the central AI-product-design lesson for B2B: be deliberate about what's AI vs deterministic. The LLM should never own a state-changing action; it should propose, summarize, parse, and translate.

23.2.5 Metrics tree



The leading indicators we watch most:

- **Onboarding completion rate** (within 30 days of trial start). Below 60% means the product is too hard to set up.
- **Reconciliation ROI proven** (within 60 days). Customer needs to see that we saved them ₹X this month. If not, churn.
- **Module adoption breadth**. Brands using 3+ modules retain at 90%+; brands using 1-2 modules retain at 50%.

23.2.6 MVP plan with kill criteria

Phase 1 (months 1-4): Build core. Inventory dashboard + Amazon/Flipkart reconciliation. Two integrations only. Closed alpha with 10 design-partner brands (free).

Phase 2 (months 5-8): Add Myntra, own-website (Shopify), demand forecasting. Open beta with 50 brands. Begin charging.

Phase 3 (months 9-12): Polish + sales motion. Self-serve onboarding. Sales team of 3. 200 brand target by EOY.

Kill criteria:

- Onboarding completion <50% after 3 cycles of UX improvement: kill.
- Customer testimonial/ROI proof under 30% of trials: kill or radically reposition.
- NRR <100% at month 12: kill (B2B SaaS doesn't survive without expansion).

23.2.7 Trade-offs accepted

1. **Integration depth > breadth.** Two marketplaces (Amazon, Flipkart) at MVP, not five. Slower expansion; better quality on what we have.
2. **Reconciliation > forecasting at MVP.** Reconciliation is the immediate ROI; forecasting is the eventual differentiator. Lead with the immediate.
3. **B2B sales motion > pure PLG.** D2C ops managers don't try-and-buy enterprise software. Need sales-assisted motion. More expensive; necessary.
4. **Annual contracts > monthly.** B2B retention is better with annual; gives us cash flow; trades ramp time for stability.
5. **No multi-tenant data sharing.** Some forecasting accuracy gain from aggregating across brands; we choose not to do this. Trust is more valuable than marginal accuracy.
6. **Slack integration > Microsoft Teams.** Indian D2C brands largely on Slack. Cuts addressable market by ~5-10% short term.
7. **English-only at MVP.** Indian D2C ops is largely English; vernacular UI is a v2 problem.
8. **Build vs buy on shipping/logistics.** We do not build shipping. Customers stay with Shiprocket, Delhivery, etc. We integrate.

23.2.8 Post-launch retrospective

Month 12, looking back:

What worked:

- Reconciliation was a killer wedge. Customers paid ₹15K/month to save 15-20 hours of manual reconciliation work. Easy ROI math.
- Design-partner motion produced 10 strong testimonials. Worth more than any marketing.
- Sales motion through founder + 1 SDR landed first 50 customers.
- Slack notifications for anomalies became a beloved feature. Sticky.

What didn't:

- Demand forecasting hit MAPE of 32% at 60-day horizon, above target. Forecasting in D2C is genuinely hard (promotional spikes, fashion-cyclical SKUs). Customers used it directionally; few trusted it for ordering decisions. Reframed as "decision support" not "automation."
- Onboarding took 3-4 weeks — slower than target. Marketplace API access requires customer to grant permissions; many didn't have admin access; back-and-forth.
- Mid-tier (\$40K) pricing tier was a dead zone. Brands at ₹5-15Cr couldn't afford it; brands at ₹15-30Cr usually wanted enterprise customization. Repackaged into two tiers (₹15K basic, ₹75K+ enterprise).
- Customer support module was used less than expected. Ops cared more about ops than about customer support. De-emphasized.

- Multi-channel pricing orchestration: legally complex (some marketplaces have anti-coordination rules); shipped a watered-down version.

What changed in v1.1-v1.4:

- Onboarding rebuilt to be 5 days, not 4 weeks. Required us to build "delegated permissions" + a guided setup flow.
- Forecasting repackaged as "decision support with confidence intervals" not "predictions."
- Pricing simplified to two tiers.
- Customer support module deprecated; integrated with Freshdesk instead.

What we'd do differently from scratch:

- Onboarding speed is product. Should have invested in 5-day onboarding from day one, not month 6.
- Pricing tier middle-out. Test with one tier, expand.
- Forecasting later. Lead with reconciliation; add forecasting once we had data.
- More design partners earlier. We had 10; should have had 30 — the variance in D2C operations is large.

23.2.9 What would Google look for here?

This is a B2B story. Different signals than Saarthi.

Strong signals:

- Customer-evidence-driven scoping (25 interviews informed the priority order).
- Architectural discipline on AI vs rules (state-changing actions are rules-based; cognition is AI).
- B2B-specific decisions visible (annual contracts, sales-assisted motion, integration depth).
- Honest about what didn't work (forecasting accuracy, mid-tier pricing).
- Pivot agility (forecasting reframed; CS module deprecated).

Likely follow-ups:

- "How would your strategy change if Shopify launched a competing operations module?" (Distribution-incumbent question.)
- "Walk me through how you'd think about expanding to Southeast Asia D2C brands." (International strategy.)
- "Your NRR target is 120%. How do you actually drive that?" (Tests B2B-specific motion knowledge.)
- "Why did you choose ₹15K basic, not ₹10K or ₹20K?" (Pricing reasoning.)
- "What's your defensibility 3 years out as more competitors enter?" (Moat thinking.)

A strong PM has specific reasoning for each. The journey above teaches the muscle.

End of Journey 2.

\newpage

JOURNEY 3: MENTORA — Tutor Marketplace for Class 9-12 Students

23.3.1 Origin and context

Indian Class 9-12 tutoring is a ₹50,000+ Cr market — heavily fragmented, mostly offline, mostly informal. Students take tuitions from local teachers, IIT/IIM-pedigree college students looking for income, or institute-employed teachers moonlighting. Discovery happens through word-of-mouth, residential WhatsApp groups, school referrals, and Justdial-style listings.

The pattern is broken in specific ways:

- **Quality variance is enormous.** A "physics tutor" can be a recent IIT grad or a person who failed their own boards. No standardized signal.
- **Pricing is opaque.** Tutors charge ₹300-₹5000/hour with little transparency on value vs price.
- **Discovery is local.** A Lucknow student is unaware that a brilliant IIT Madras student would teach online for ₹500/hour.
- **Trust is broken.** No reviews, no track record, no recourse if it doesn't work.
- **Scheduling is informal.** Phone calls, cancellations, no-shows.

Existing online tutoring platforms (Vedantu Live, Unacademy Plus, Tutorial Point) are content-heavy or institute-style — not marketplaces of independent tutors. The pattern that works for adult skills (Preply, italki) hasn't been built for Indian school tutoring.

The hypothesis: a marketplace connecting Class 9-12 students with vetted tutors — quality-signaled, transparently priced, with payments and scheduling handled — could capture a meaningful slice of this market.

The opening question (very different from Saarthi's): **how do you cold-start a two-sided marketplace?** Marketplaces fail more often than they succeed. The dynamics are unforgiving: without enough students, tutors leave; without enough tutors, students don't engage. The "chicken and egg" problem is the central PM challenge.

23.3.2 Discovery — both sides of the marketplace

Marketplaces require two-sided discovery. Twenty-five interviews each:

Student/parent side (25):

- 15 students currently in tuitions.
- 10 parents.

Tutor side (25):

- 10 IIT/NIT/IIM students currently tutoring.
- 8 full-time tutors / school teachers moonlighting.
- 7 institute-employed tutors.

Student/parent side — what we learned

Pain 1: Discovery friction. "I found my tutor through a neighbor's recommendation. I have no way to know if there's someone better." (18 of 25.)

Pain 2: Quality calibration. "How do I know this tutor is actually good before paying?" Trial classes are common but inconsistent.

Pain 3: Travel cost. Tier-1 metros: tutors come home, charge premium. Tier-2: students travel to tutor; takes 30-45 min each way; eats study time.

Pain 4: Schedule unreliability. Tutors cancel, reschedule, ghost. No accountability.

Pain 5: Pricing opacity. Different tutors charge different rates without obvious reason. Hard to know if price is fair.

What students/parents wanted but probably don't:

- "More content" — there's already too much content. Not the issue.
- "AI tutors" — most parents are skeptical. Want human tutors with AI assistance, not AI replacements.

Tutor side — what we learned

Pain 1: Inconsistent income. "I have 4 students. If 2 cancel for exams, I lose half my month's income."

Pain 2: Discovery friction (mirror). "I'd take more students. I just don't know how to find them."

Pain 3: Payment hassle. "Some parents pay late. I can't ask my IIT batchmate's mom for ₹3000 every month aggressively."

Pain 4: Time wasted on logistics. "I spend an hour a week scheduling, an hour on commuting if it's offline, an hour on chasing payments. That's 3 hours I could teach."

Pain 5: No career progression. "Tuition is a side hustle. I have no track record I can take to grow into a teacher or content creator."

What tutors said they wanted that they probably don't:

- "More features" — what they actually want is more students at fair rates.

Synthesis: the JTBDs (two-sided)

Student/parent JTBD:

"When my child needs help with [subject/topic], I want to find a vetted, quality tutor at a fair price who shows up reliably and helps my child improve — without the friction of word-of-mouth searching, awkward payment chasing, or unreliable scheduling."

Tutor JTBD:

"When I want to teach for income, I want predictable bookings, fair pay, on-time payment, and a track record I can build — without spending half my time on logistics and chasing students."

The product has to serve both, simultaneously. The marketplace lives or dies by serving the harder side better.

23.3.3 The cold start strategy — the hardest decision

For a marketplace, *which side do you build first?*

Three strategies considered:

Option A: Tutors first, then students. Recruit 100 high-quality tutors. Promise them students. Then market to students with "100 vetted tutors waiting."

Risk: tutors leave if students don't show up fast. Their patience is ~4-8 weeks.

Option B: Students first, then tutors. Build a waitlist of students. When you have 500, recruit tutors aggressively — "500 students looking for tutors right now."

Risk: students leave if tutors aren't available immediately. Their patience is ~1-2 weeks.

Option C: Both, in narrow geography/subject focus. Pick one subject (Physics) and one geography (one Bangalore neighborhood + Class 11-12). Build dense supply and demand simultaneously.

Risk: slow expansion; addressable market in any one cell is limited.

Decision: Option C.

Reasoning:

- Marketplaces succeed locally first, then expand. Doordash started in Palo Alto. Airbnb started in NY. Trying to be national from day one almost always fails.
- Density matters. A marketplace where students see "5 physics tutors near me" feels usable. "1 physics tutor near you, 4 in another city" doesn't.
- Subject focus lets us tune the matching algorithm and quality signals before expanding.

The wedge: Class 11-12 Physics, Bangalore HSR/Koramangala/Indiranagar, ₹500-1500/hour range. We aim for 30 active tutors and 200 active students in this cell within 4 months.

23.3.4 The PRD (compressed)

Product: Mentora — Marketplace connecting Class 9-12 students with vetted tutors, online and offline.

Core flow:

1. Parent/student creates account, lists subject and grade.
2. Browses or gets recommended tutors with profiles (background, ratings, sample lessons, price).
3. Books a free 30-minute trial class.
4. After trial, books recurring sessions (1x or 2x/week).
5. Mentora handles payments — student pays Mentora; Mentora pays tutor weekly minus 15% commission.
6. Both rate each other after each session.
7. Issues (cancellation, dispute, quality concern) escalated to Mentora support.

Vetting (the trust spine):

- All tutors verified: ID, education credentials, sample teaching video.
- IIT/NIT/IIM students get a "verified institute" badge.
- All tutors take a subject competency test before listing.
- First 10 sessions monitored for quality (random sampling); poor-quality tutors removed.

Pricing structure:

- Free for students/parents to use.
- 15% commission from tutor on each booking.
- Premium tutor tiers: 12% commission for tutors with 4.8+ rating and 50+ sessions.

Success metrics:

- Active student-tutor pairs (defined as 3+ sessions in a 30-day window).
- Match rate (% of students who book a session within 7 days of joining).
- Trial-to-paid conversion (% of trials that become recurring).

- Tutor retention at 3 months.
- NPS, both sides.

Non-goals:

- We are not building tutor content/curriculum.
- We are not a coaching institute.
- We are not handling university/college tutoring.
- We do not own homework completion or test-taking.

23.3.5 Technical architecture (briefly)

Marketplace platforms have a recurring architecture:

- **Discovery:** search + recommendation. ML-driven matching based on student needs and tutor profile.
- **Trust layer:** verification workflows, ratings system, dispute handling.
- **Booking:** scheduling, calendars, video integration (for online sessions).
- **Payments:** escrow model — student pays upfront; held; paid to tutor on session completion.
- **Communication:** in-app chat (do not let conversations move off-platform; otherwise disintermediation kills marketplace).

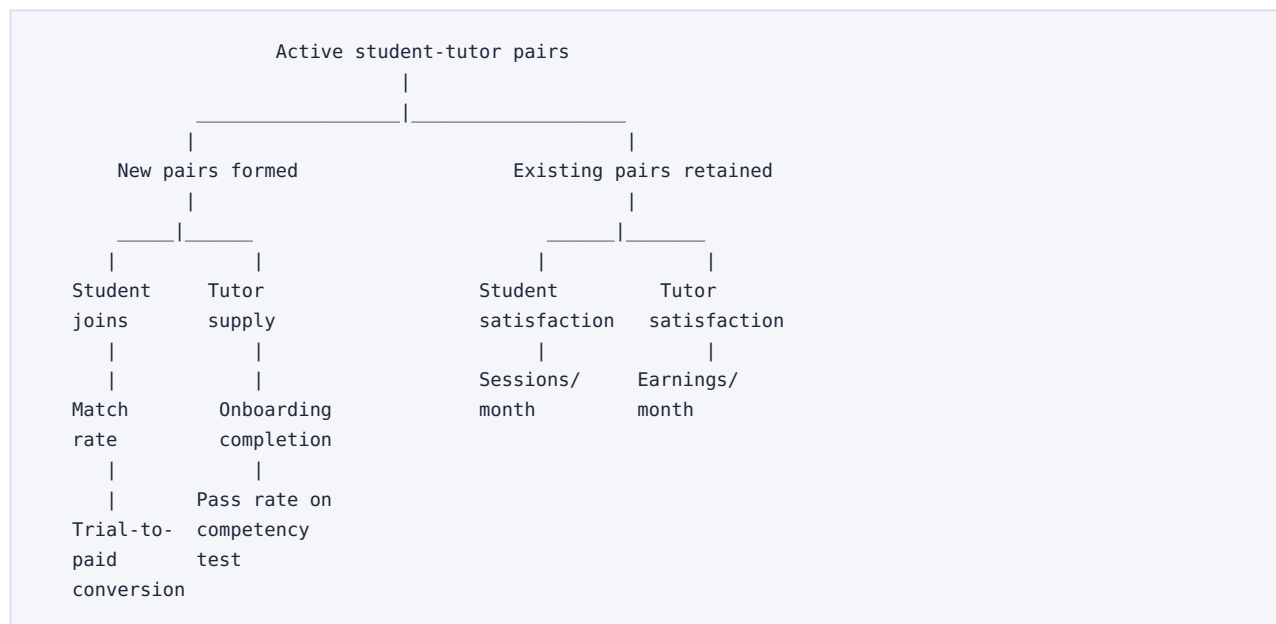
Where AI sits (in 2026):

- **Matching.** When student lists subject + needs, AI suggests tutors based on profile match, rating, recency, availability.
- **Quality detection.** AI analyzes (with consent) recordings from sample sessions. Flags low-quality teaching for review.
- **Dispute summarization.** When students/parents file disputes, AI summarizes the conversation history for support agents.
- **Tutor onboarding.** AI helps tutors write their profiles, generates sample teaching videos with feedback.
- **Practice problem generation.** AI generates practice problems calibrated to the student's level — used by tutors during sessions.

Where AI does NOT sit:

- Vetting/credential verification (humans + ID/document checks).
- Dispute resolution (humans make final call).
- Pricing recommendations (rules-based + market data).
- Payments (deterministic, audited).

23.3.6 The marketplace metrics tree



The two killer leading indicators:

1. **Time-to-first-session for new students.** If <5 days, retention is high. If >10 days, students drop off.
2. **Tutor utilization rate (% of tutor's listed availability that's booked).** If <30%, tutors leave within 2 months. If >60%, tutors stay and recruit other tutors.

23.3.7 MVP plan with kill criteria

Phase 1 (months 1-4): The wedge

- Build core platform.
- Recruit 30 Physics tutors in Bangalore (HSR/Koramangala/Indiranagar).
- Recruit 200 Class 11-12 students in same area.
- Heavy white-glove onboarding for both sides.
- Free trials, manual matching, hands-on quality monitoring.

Phase 2 (months 5-8): Validate + expand subject

- Add Chemistry, Math (still Bangalore Class 11-12).
- Move from manual matching to algorithmic.
- Test commission structure changes.
- 800 active student-tutor pairs target.

Phase 3 (months 9-12): Geographic expansion

- Hyderabad, Chennai, Pune.
- Expand grade range to Class 9-10.

- Self-serve onboarding for tutors.
- 3000+ active pairs target.

Kill criteria:

- If tutor utilization stays below 30% for 3 months: kill or radical rethink.
- If trial-to-paid conversion below 20%: tutors aren't good enough or matching isn't working.
- If commission revenue at month 12 below ₹50L MRR: unit economics broken.
- If NPS gap between sides is wide (e.g., students 60, tutors 20): something is breaking on the tutor side; fix or fail.

23.3.8 Trade-offs accepted

1. **Geographic narrowness > national.** Slower growth; better marketplace dynamics.
2. **Quality > quantity of tutors.** Vetting friction reduces tutor count; raises quality bar.
3. **Online + offline > online only.** Indian parents trust offline more; we can't be online-only at MVP.
4. **15% commission > lower.** Lower commission would be more attractive to tutors but kills unit economics.
5. **Payment escrow > direct.** Slower to tutor; necessary for trust.
6. **In-app comms > free messaging.** Some students/tutors will move offline; we accept some leakage; protecting the marketplace requires friction.
7. **Single subject at a time > breadth.** Slower addressable market; cleaner matching.
8. **Rating system > algorithm-only.** Public ratings are vulnerable to gaming; we accept some manipulation to preserve trust signals.

23.3.9 Post-launch retrospective

End of year 1, looking back:**What worked:**

- Geographic narrowness paid off. Bangalore Phase 1 hit 280 active pairs by month 5; word-of-mouth was strong because density was real.
- Vetting was a marketing asset. Parents valued the "verified" badge above almost everything else.
- Trial class conversion was 38% — above target. Free trial removed first-purchase friction.
- IIT/NIT student tutors specifically were a magnet. They charged premium and parents paid willingly.

What didn't:

- Online sessions had higher cancellation rates than offline. Students treated online as lower-commitment. Adjusted: required a small commitment fee for online trials.
- Tutor utilization rate was 36% in early months — borderline. Many tutors complained. We had to subsidize early tutors with bonus payments to keep them.
- Disintermediation was real. About 12% of student-tutor pairs eventually moved off-platform after 4-5 sessions. Reduced through: better integrated payments (frictionless), session recording features (quality value-add), homework tracking.
- Class 9-10 students were a different segment than Class 11-12. Less self-driven, more parent-mediated, lower price tolerance. Required different onboarding.
- Geographic expansion to Hyderabad/Chennai was harder than expected. Each city required its own dense supply build-out. We learned: marketplaces are fundamentally local.

What we changed in v1.x:

- Online sessions: required commitment deposit; reduced cancellation.
- Subsidies: programmatic for first 90 days of new tutor; data-driven thresholds.
- Disintermediation: integrated session recording, homework grading, progress reports — features that have value only on-platform.
- Class 9-10: separate "parent-friendly" UX track with simpler matching, more guidance.
- Expansion: shifted from "launch a city in 30 days" to "launch a neighborhood in 60 days, then expand."

What we'd do differently from scratch:

- Subsidies built into the model from day one. We were too late realizing tutor utilization required intervention.
- Online vs offline as separate products. Different unit economics, different user expectations. Bundling them was a complexity tax.
- Parent-side product earlier. We optimized for student users; parents are the buyers.

23.3.10 What would Google look for here?

Marketplace product stories test specific muscles:

Strong signals:

- Cold-start strategy with explicit reasoning. Local density first; not "national from day one."
- Two-sided JTBD. Clear understanding that supply and demand are different products.
- Trust infrastructure as core. Vetting, payments, ratings are the spine.
- Disintermediation awareness. PM understands the marketplace's mortal threat.
- Unit economics rigor. Commission structure debated explicitly.

Likely follow-ups:

- "What if a major coaching institute (Allen, Aakash) launched a competing marketplace?" (Strategic competition.)
- "How would you decide when to expand to which city?" (Geographic strategy.)
- "Walk me through how you'd think about international expansion — say, the US tutoring market." (Strategic thinking on adjacent markets.)
- "Your trial-to-paid conversion is 38%. What's the highest-leverage lift here?" (Funnel optimization.)
- "How do you think about the trade-off between protecting the marketplace and giving users freedom?" (Marketplace philosophy.)

A strong PM has reasoned answers. The journey above is a starting point.

End of Journey 3.

\newpage

JOURNEY 4: VEGA — Internal AI Eval Platform at a Mid-Stage Fintech

23.4.1 Origin and context

This journey is different from the previous three — it's an **internal product**. The PM is not building for external customers; they're building for engineering and ML teams inside their own company.

Internal products are often more interesting than external ones because the dynamics are different:

- Your customers are colleagues. They can't churn easily; they also have political weight.
- Your "competitors" are: build-it-yourself, buy from vendor (e.g., LangSmith, Braintrust, Arize), or do nothing.
- ROI is measured in engineering productivity and risk reduction, not revenue.
- Failure is silent. People just stop using your tool, but they won't say it directly.

The context: a mid-stage Indian fintech (~₹500Cr revenue, ~150 engineers, ~25 ML/AI engineers). Over the past 18 months, the company has shipped 12 LLM-powered features — fraud detection summaries, customer support copilot, document parsing for KYC, transaction descriptions, etc. Each feature has its own ad-hoc evaluation: a Jupyter notebook, a manual sample, a custom dashboard.

The pattern is unsustainable:

- No one knows which feature's eval is current vs stale.
- Different teams use different metrics; comparisons are impossible.
- Regression detection is reactive (a customer complaint triggers investigation).
- Onboarding new ML engineers requires re-explaining eval setup every time.
- Compliance/risk asks: "Show us the eval coverage of every LLM-touching feature."
Nobody can answer.

The hypothesis: a unified internal evaluation platform — Vega — that provides a standard way to define, run, and monitor AI evals across the company would meaningfully improve AI feature quality and reduce risk.

23.4.2 Discovery — internal interviews

For internal products, "discovery" is interviewing colleagues. Twenty interviews:

- 10 ML/AI engineers building LLM features.
- 4 engineering managers.
- 3 from risk/compliance.
- 2 from product (PMs of AI features).
- 1 CTO.

What we asked:

- "Walk me through how you currently evaluate the LLM feature you're working on."
- "When was the last time you caught a regression before it shipped? When was the last time you missed one?"
- "If a regulator asked tomorrow how you ensure your AI features are safe, how would you answer?"
- "What's the most painful part of your eval workflow?"

What we learned:

Pain 1: Eval set fragmentation. "Every feature has its own eval set. Some have 50 examples; some have 500. Some are versioned; some aren't. Some are in Jupyter; some in Notion." (8 of 10 engineers.)

Pain 2: No regression detection. "We added a new prompt last month. We didn't run the old eval set against the new prompt because that would take half a day. Two weeks later, support got complaints about a regression we'd introduced." (5 of 10 engineers — and these are people willing to admit it.)

Pain 3: No production monitoring. "We track latency and error rate in production. We don't track quality. We have no idea if the model started hallucinating last Tuesday." (9 of 10 engineers.)

Pain 4: Cross-team blindness. EMs: "I have 5 engineers shipping LLM features. I don't know which has good evals and which has none. I'm flying blind on quality."

Pain 5: Compliance anxiety. Risk/compliance: "Regulators are starting to ask about AI governance. Right now we have a few PowerPoint slides. We'd be in trouble if there's an incident."

Pain 6: Onboarding cost. "Every new ML engineer takes 2-3 weeks to set up their eval workflow because there's no shared infrastructure. That's tax we keep paying."

What we didn't expect

Several engineers expressed a *preference* for the status quo:

- "I have my eval the way I like it. A platform would impose constraints."
- "I don't want my evals visible to other teams. If someone sees a 75% pass rate, they'll think my feature is bad."
- "I don't want compliance breathing down my neck."

This is the platform-PM problem. **Internal users often resist standardization, especially when it brings visibility.** Vega will face active resistance from some users, even though leadership wants the platform. Navigating this is the central PM challenge.

JTBD synthesis

Two JTBDs, in tension:

ML engineer JTBD:

"When I'm shipping or maintaining an AI feature, I want fast eval feedback so I know if my changes improved or regressed quality — without a heavy infrastructure burden, and without exposing my work to unwanted scrutiny."

Leadership/compliance JTBD:

"When I'm responsible for AI quality and risk across the company, I want unified visibility into eval coverage, regression detection, and quality trends — so I can answer 'are we safe?' with evidence."

These two jobs sometimes align (good evals serve both) and sometimes conflict (engineers don't want their numbers visible). The PM has to design for both, with explicit visibility controls.

23.4.3 The PRD (compressed)

Product: Vega — Internal AI evaluation platform.

Core capabilities:

1. **Eval set management.** Versioned eval sets stored centrally. CSV/JSONL upload, programmatic creation, organic growth from production logs.
2. **Eval runner.** Run any eval set against any LLM/prompt configuration. Cached results. Compare runs side-by-side.
3. **LLM-as-judge framework.** Out-of-the-box judges for common dimensions (correctness, toxicity, format adherence). Custom judges easy to create.
4. **CI/CD integration.** Plug into existing GitHub Actions / Jenkins. Eval runs are part of the deploy pipeline.
5. **Production monitoring.** Sample production traffic; run shadow evals; alert on quality drops.
6. **Dashboards and reporting.** Per-feature views, cross-feature comparisons, executive summary.
7. **Visibility controls.** Eval results visible to: feature team only / engineering org / leadership. Configurable. Engineers control their visibility.

Primary users:

- Tier 1: ML/AI engineers (most direct usage).
- Tier 2: EMs of teams shipping AI features.
- Tier 3: Risk/compliance, leadership.

"Pricing": No external pricing. Internal cost: 1 PM, 4 engineers, 1 designer, ~12 months to v1. Approximate cost ~₹5Cr fully loaded.

Success metrics:

- Adoption: % of LLM-touching features in the company using Vega.
- Eval coverage: % of production LLM calls with active eval coverage.
- Time to detect regression (MTTD): from change deployed to regression flagged.
- Engineer satisfaction (NPS among engineers using Vega).

Non-goals:

- We are not building model training infrastructure.
- We are not building inference infrastructure (that's another team's domain).
- We are not building a product analytics platform (Mixpanel etc. own that).
- We are not handling RLHF / fine-tuning data labeling at MVP.

23.4.4 Architecture decisions

Build vs Buy vs Hybrid

The hardest strategic question for an internal platform.

Buy options (in 2026):

- LangSmith (LangChain's offering).
- Braintrust.
- Arize Phoenix.
- Weights & Biases (Weave).
- Helicone.

Build option: internal platform from scratch.

Hybrid: integrate one of the above as the engine; build internal layer for our specifics.

Decision: Hybrid, leaning toward Braintrust as engine + internal wrapper.

Reasoning:

- Building eval infrastructure from scratch is 12-18 months of work. Most of it is undifferentiated (eval runner, score storage, dashboard).
- Vendors do this part well. They've solved problems we'd take years to solve.
- The differentiating layer is: integration with our internal systems (CI/CD, monitoring, compliance reporting), our specific eval patterns, our org-specific UX and governance.
- Braintrust specifically: strong API, good multi-tenant model, reasonable pricing.

Trade-off accepted: vendor lock-in. If Braintrust changes pricing, sunsets a feature, or gets acquired, we have to migrate. Mitigation: keep our wrapper layer abstract; design for substitutability.

LLM-as-judge architecture

Most evaluations in 2026 use LLM-as-judge for subjective dimensions (quality, helpfulness, tone). The architecture:

- Judge LLM (typically a frontier model — GPT-4o, Claude Sonnet) scores outputs on each dimension.
- Multiple judges available; engineers choose based on need.
- Calibration: 5-10% of judge outputs randomly sampled for human review; judge prompt revised when human-judge disagreement is high.

The risk with LLM-as-judge: drift. The judge model itself updates; scores shift; what looks like a regression might just be the judge changing its mind.

Mitigation: pin judge model versions; rotate carefully; track judge consistency over time.

Production monitoring

Three patterns:

1. **Shadow mode.** Run new configurations against production traffic without serving the result. Compare quality.
2. **Canary mode.** Serve new configuration to 5% of traffic; compare quality with control.
3. **Full rollout monitoring.** After 100% rollout, continuously sample and run quality checks.

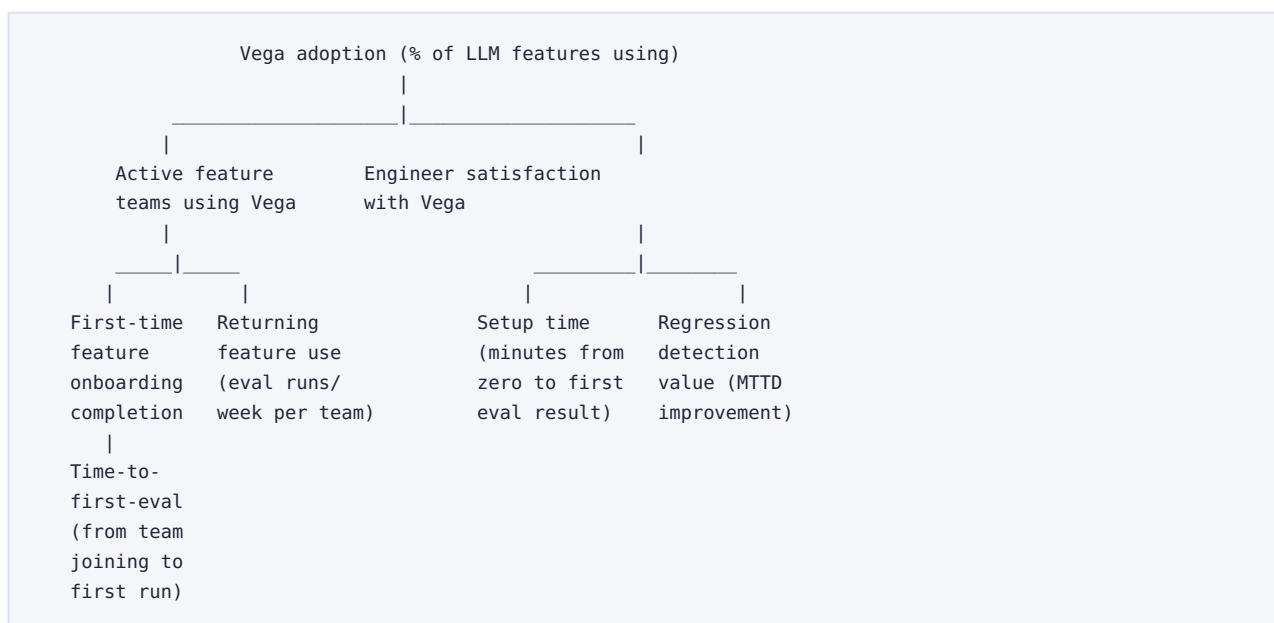
All three have value at different lifecycle stages. Vega supports all.

Storage and data architecture

Eval data is sensitive. Specific decisions:

- Eval results stored in our data warehouse (Snowflake), not Braintrust's infrastructure.
- Eval inputs/outputs: encrypted at rest. Access-controlled by team.
- Production samples: redacted of PII before being stored as eval data.
- Audit log for all eval data access (compliance requirement).

23.4.5 The metrics tree



The leading indicator we watch most: **time-to-first-eval for new feature teams**. If it takes >2 hours from "I want to use Vega" to "first eval result," teams give up. Target: <30 minutes.

The killer adoption signal: feature teams *recruiting* other teams to use Vega. When that happens, we've crossed the chasm internally.

23.4.6 The MVP plan

Phase 1 (months 1-4): Core platform

- Eval set management.
- Eval runner with Braintrust integration.
- 2-3 judge templates.
- Per-feature dashboards.
- 3 design-partner teams (the most willing internal customers).

Phase 2 (months 5-8): Production monitoring + CI/CD

- Production sampling.
- Shadow mode.
- GitHub Actions integration.
- 8-12 teams using.

Phase 3 (months 9-12): Compliance + scale

- Org-wide dashboards for leadership/compliance.
- Cross-team comparisons (carefully, with visibility controls).
- Self-serve onboarding for any team.
- 80%+ adoption target.

Kill criteria:

- Below 50% adoption among AI-shipping teams by month 12: kill or radical rethink (perhaps deprecate in favor of pure vendor).
- Engineer NPS below 0 (more detractors than promoters) consistently: kill or rebuild.
- Regression-detection MTTD doesn't improve materially: tool isn't doing its job.

23.4.7 Trade-offs accepted

1. **Hybrid (vendor + wrapper) > pure build or pure buy.** Faster ship; some lock-in.
2. **Engineer-controlled visibility > leadership-driven visibility.** Engineers will adopt; leadership gets the dashboards they need eventually, not on day one. Sequence matters.
3. **Pull > push for adoption.** No mandate from CTO that teams must use Vega. Adoption earned. Slower; durable.
4. **Self-service > white-glove for v1.** Some teams will struggle; we accept that to focus on the path-to-scale.
5. **Templates > full customization at MVP.** Limited eval template options; expand once we know what teams need.
6. **Production sampling > live evaluation.** Sampling is cheaper, sufficient for monitoring; live evaluation is too expensive at scale.

7. **Plain dashboards > beautiful design at MVP.** Internal product; engineers tolerate ugly; we save design budget for the parts that matter.
8. **Engineering-first > product-first language.** UX speaks engineering language (PRs, configs, metrics). Less friction for the target user; less accessible to non-engineers; intentional.

23.4.8 Post-launch retrospective

Year 1, looking back:

What worked:

- Hybrid architecture was right. We shipped MVP in 4 months instead of 12 by leveraging Braintrust.
- Design-partner teams generated genuine word-of-mouth. By month 6, teams were asking us for access; we didn't have to recruit.
- CI/CD integration was sticky. Once a team set up GitHub Actions to run evals on PRs, removing it was harder than keeping it.
- Production monitoring caught 3 real regressions in the first 6 months that would have shipped to customers.

What didn't:

- Adoption stalled at ~55% by month 12. Some teams resisted (especially older teams with established workflows). Cultural change takes time.
- Cross-team comparison dashboards launched in Phase 3 caused an uproar — some teams felt exposed. We made comparison opt-in only.
- Compliance reporting was harder than expected. Each compliance question had nuances we didn't anticipate. Required custom reports per request.
- LLM-as-judge drift was real. Judge model updates caused score shifts that looked like regressions but weren't. Took 3 months of investigation to recognize the pattern.
- Some teams gamed evals — added easy questions to their eval set to inflate pass rates. We added "eval set diversity" metrics; partially solved.

What we changed in v1.x:

- Made cross-team dashboards opt-in. Restored team trust.
- Built "judge model snapshot" feature — pin judge to specific version per eval set.
- Started weekly office hours for teams; helped with adoption.
- Built self-serve "starter eval set" for common feature types.
- Added eval set diversity scoring.

What we'd do differently:

- Engagement strategy with non-believer teams sooner. We focused on the willing teams; should have invested in winning over the skeptical ones earlier.
- Compliance partnership earlier. Should have had compliance team co-design the dashboards; would have saved rework.
- Judge model versioning from day one. Lesson learned from production drift.
- More explicit "this won't be visible to others" messaging early. Trust mattered more than we expected.

23.4.9 What would Google look for here?

Internal-platform PM is a different beast. Strong signals:

- **Awareness of the build-buy-hybrid tradeoff.** This is a senior PM signal.
- **User segmentation across multiple stakeholder layers.** Engineers vs EMs vs leadership vs compliance — all customers.
- **Trust-aware design.** The visibility-control discussion shows political awareness.
- **Adoption-led metrics.** Pull > push; word-of-mouth as adoption measure.
- **Honest about resistance.** "Some teams resisted" is a real platform-PM truth.

Likely follow-ups:

- "Some teams are still resisting at month 12. What's your strategy to win them over?" (Influence under resistance.)
- "If a senior leader mandates Vega usage, do you support that?" (Tests philosophy on top-down adoption.)
- "How do you decide when to deprecate features that aren't gaining traction?" (Platform discipline.)
- "Walk me through how you'd think about open-sourcing parts of Vega." (Strategic thinking on internal-vs-external products.)
- "What's the long-term position of internal AI tooling vs vendor solutions?" (Build-vs-buy strategic horizon.)

This kind of internal-platform thinking is rare among early-career PMs. If you can speak fluently to it, you signal senior-grade thinking.

End of Journey 4.

\newpage

JOURNEY 5: KARYA — Autonomous Email & Calendar Agent for Knowledge Workers

23.5.1 Origin and context

This is the most ambitious of the five journeys. Karya is an autonomous AI agent — meaning it doesn't just suggest actions, it takes them. It reads your email, schedules your meetings, drafts replies, books travel, and follows up — all with minimal supervision once trusted.

Why this is hard:

- **Trust is the binding constraint.** Users won't let an AI take actions on their behalf until they trust it. Trust takes weeks to build, can be destroyed in one bad action.
- **The agent has multiple interaction surfaces.** Email, calendar, sometimes Slack, sometimes Notion. Each integration adds complexity and risk.
- **Autonomy UX is not solved.** "What do I let the agent do without asking?" has no industry-standard answer.
- **Recovery from failure is critical.** When the agent gets it wrong (and it will), what happens?

The market context: by 2026, AI agents have become a popular product category. Cognition Devin (coding agent), Adept ACT-2, Multion, OpenAI's Operator, and several incumbent moves (Microsoft Copilot, Google Duet) have established the pattern. None have nailed personal productivity (email + calendar) for knowledge workers.

The hypothesis: an agent specifically designed for knowledge-worker email and calendar — with a deliberate trust-building UX and strong defaults — could win this category.

23.5.2 Discovery — power-user interviews

For this product, the target user is sharp: knowledge workers (PMs, founders, executives, consultants) who deal with 100+ emails/day and 8-15 meetings/day.

Twenty interviews:

- 8 startup founders / CEOs.
- 5 senior PMs / VPs at tech companies.
- 4 management consultants.
- 3 executive assistants (who currently do this job manually).

What we asked:

- "Walk me through your typical email morning."

- "What's your worst recurring email/calendar task?"
- "If you had a perfect assistant, what would they do?"
- "What would terrify you about an AI handling your email?"

What we learned:

Pain 1: Inbox triage exhausts cognitive capacity. "I spend 90 minutes a day deleting, archiving, and triaging email. Most of it is junk. The signal is buried in noise." (16 of 20.)

Pain 2: Scheduling back-and-forth is brutal. "Setting up a 30-min meeting takes 6 emails. Multiply by 15 meetings/week." (14 of 20.)

Pain 3: Follow-ups slip. "I owe 8 people responses. I forget. They get frustrated." (12 of 20.)

Pain 4: Drafting routine replies takes time. Confirmations, polite declines, intro responses. "I'm rewriting the same email 20 times a week." (15 of 20.)

Pain 5: Travel booking is friction-heavy. "Three emails with the executive assistant + browsing Cleartrip + checking calendar. Hours per trip." (8 of 20, mostly executives.)

The trust question — most surprising finding:

Nearly every interviewee, when asked "what would terrify you about AI on your email," gave the same set of answers:

- "Sending an email I would never have sent." (Tone errors, premature commitments.)
- "Sharing private information." (Forwarding the wrong thing; replying-all by mistake.)
- "Deleting something I needed." (Aggressive triage.)
- "Booking a meeting I didn't want." (Calendar invite to someone the user was avoiding.)
- "Missing something important." (Filing it as junk by mistake.)

These are **specific** fears. The agent must address each, individually, in its design.

What people said they wanted that they probably don't:

- "An AI that just handles everything autonomously." Most users *say* this; they don't actually want it. They want oversight with leverage. Pure autonomy is too scary at v1.

JTBD synthesis

"When I'm overwhelmed with email and scheduling, I want an AI assistant that handles routine tasks and surfaces only what genuinely needs my attention — without the fear that it will say something wrong, leak private information, or miss something important — so I can spend my cognitive energy on actual work."

The job is "reduce cognitive load while preserving control." Both halves matter.

23.5.3 The PRD (compressed)

Product: Karya — Personal AI agent for email and calendar.

Core capabilities (in order of trust difficulty):

1. **Triage (read-only).** Karya reads email and categorizes: must-respond, can-wait, junk. No actions taken.
2. **Drafts.** Karya drafts replies; user reviews and sends.
3. **Auto-reply for low-risk emails.** Things like "thanks, will do" and meeting confirmations. User pre-approves the categories.
4. **Calendar coordination.** Karya proposes meeting times; user approves. After trust, Karya can confirm directly.
5. **Follow-up tracking.** Karya watches for unreplied threads where user committed to respond. Reminds. Drafts.
6. **Travel and logistics.** Books flights, hotels per user preferences. Always confirms before payment.
7. **Long-term: agentic workflows.** "Find me 3 candidates with these qualifications and schedule intros." Multi-step autonomous work.

The MVP is capabilities 1-4. Capabilities 5-6 in v1.1. Capability 7 is v2+.

Primary users: Knowledge workers receiving 50+ emails/day, with 5+ meetings/day. Initial target: founders, executives, senior managers in tech.

Pricing: \$30/month consumer; \$50/user/month team. Premium tier (\$100/month) for executives with higher message volume and travel features.

Success metrics:

- Time saved per user (self-reported and behavioral measures).
- Trust progression (user enables progressively more autonomy).
- Action accuracy (errors per 1000 actions taken).
- Net Promoter Score.
- Conversion: free trial → paid.

Non-goals:

- We are not a productivity OS. We don't replace Notion, Asana.
- We are not a CRM. We don't track customer relationships systematically.
- We are not a calendaring app. We work with Google Calendar / Outlook; we don't replace them.
- We are not a writing assistant. We help with email drafts; we're not a Grammarly.

23.5.4 The trust progression UX — the most important design decision

The single biggest design choice: how does the agent earn autonomy?

The model: progressive trust through demonstrated reliability.

Phase 0 (week 1): Observation. Karya does nothing. It reads the user's email and calendar to learn patterns. Shows insights: "I noticed you respond to investor emails within 4 hours, internal team within 24 hours." Builds user understanding of what Karya sees.

Phase 1 (weeks 2-4): Suggestions only. Karya suggests actions but doesn't take any. "Here's a draft reply to John's email." User reviews; sends or rejects. Karya learns from acceptance/rejection patterns.

Phase 2 (weeks 5-8): Auto-actions for low-risk categories. User explicitly enables specific categories for autonomy: "auto-confirm meeting requests from my team," "auto-archive newsletters," "auto-decline cold outreach." Karya does these without asking; user audits.

Phase 3 (weeks 9+): Expanded autonomy. User progressively enables more categories. "Schedule meetings up to 30 minutes with my team without asking." "Send routine status updates."

Phase 4 (months 3+): Full agentic. User trusts Karya enough that the dashboard shows: "Today Karya handled 23 emails, scheduled 5 meetings, sent 3 follow-ups. Here's the audit log."

The progression takes time — typically 2-3 months for full trust. We don't rush it.

The boundary system

Within each phase, the user has explicit boundary controls:

- **People allowlists/blocklists.** "Always ask before responding to my CEO." "Never auto-respond to my therapist."
- **Topic boundaries.** "Don't auto-respond about pricing." "Don't book meetings about [sensitive topic]."
- **Action boundaries.** "Never delete email." "Always show me drafts before sending if amount > \$1000."
- **Time-of-day boundaries.** "Don't take any actions after 8pm."
- **Confirmation thresholds.** "Confirm before any action involving more than 5 people."

The boundary system is the reassurance that prevents the user's worst-case fears.

The audit log

Every action Karya takes is logged. The audit log is searchable, filterable, and prominent. Users can:

- See exactly what Karya did and when.
- Undo actions (within a window — replies that haven't been read can be unsent; meetings can be uncreated).
- Provide feedback: "This action was wrong" — Karya learns.

The audit log is not buried in settings. It's a primary surface.

23.5.5 AI architecture

Models:

- Frontier model for understanding context, drafting nuanced replies, edge cases (Claude Opus or GPT-4o).
- Smaller, faster model for routine triage (Claude Haiku or GPT-4o-mini).
- Embedding model for personal context (relationship history, prior emails).

RAG over user's data:

- Index of all the user's email history (with privacy controls).
- Index of contacts and relationship context.
- Index of past actions and their outcomes (success/correction).

Memory:

- Long-term: relationship context. "Sara is my co-founder. Casual tone. Respond fast."
- Medium-term: ongoing threads. "I owe John a response on the Q3 plan."
- Short-term: today's context. "User is in meetings 2-5pm; flag urgent emails."

Action layer:

- Defined set of actions (send email, archive, schedule, etc.).
- Each action has structured parameters and validation rules.
- Action execution gated by permission system.

Safety layers:

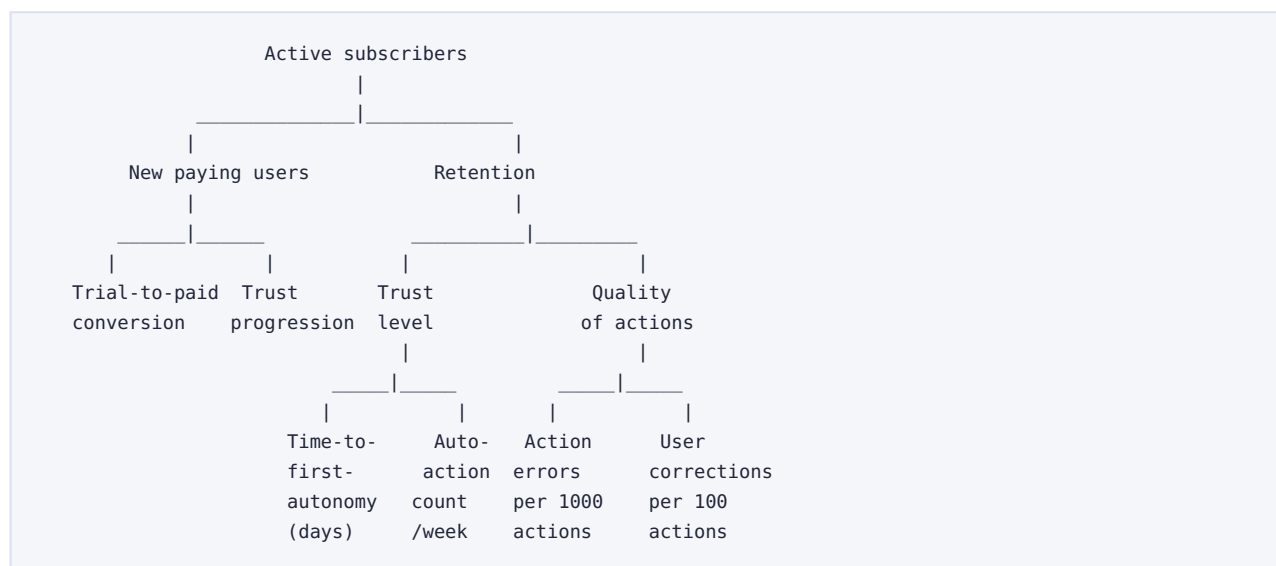
- Pre-flight checks: before any auto-action, run safety check (is this email to a sensitive contact? Does it commit to something? Does it share private info?).
- Reasoning trace: for complex actions, the agent shows its reasoning so the user can audit.
- Failure recovery: if an action fails or seems suspicious, the agent escalates to user.

Privacy architecture:

- All personal data stays in user-isolated storage.
- No training on user data without explicit consent.
- End-to-end encryption for at-rest storage.
- Clear data deletion controls.

This is essential. Email is the most sensitive workplace data. A privacy violation kills the company.

23.5.6 The metrics tree



The killer metric: **action error rate per 1000 actions**. Below 1: trust grows. Above 5: trust collapses.

The trust progression metric: **% of users at Phase 3 by week 8**. Tells us if our progression model is working.

23.5.7 The MVP plan

Phase 1 (months 1-4): Trust-building MVP

- Capabilities 1-3 (triage, drafts, auto-reply for low-risk).
- Email integration (Gmail first; Outlook in v1.1).
- Closed alpha with 50 power users (founders, senior PMs).
- Heavy white-glove onboarding.

Phase 2 (months 5-8): Calendar + follow-up

- Capability 4 (calendar coordination).
- Capability 5 (follow-up tracking).

- Public beta. 1000 users target.

Phase 3 (months 9-12): Travel + scale

- Capability 6 (travel booking).
- Self-serve onboarding.
- 10,000 user target.

Kill criteria:

- Action error rate above 2% sustained: kill or rebuild safety layers.
- User trust progression: <40% reach Phase 2 by week 6: UX or messaging issue; investigate.
- Conversion (trial → paid) below 12%: pricing or value-prop issue.
- Negative incident with PR coverage: pause feature, investigate, transparent communication.

23.5.8 Trade-offs accepted

1. **Slow trust > fast value.** We don't rush the user into autonomy. Slower perceived value early; more durable retention.
2. **Frontier model > cheaper alternatives.** Email tone errors are intolerable. We pay for quality.
3. **Read access default; write access progressive.** Karya can read everything from day one (after permission); writing actions are gated. Trust asymmetry intentional.
4. **Audit log prominent > buried.** The "what did Karya do today" surface is primary. Users need it.
5. **Power-user focus > broad consumer.** Power users have the volume that makes the value proposition clear. We win them first.
6. **Gmail first > all-or-nothing.** Half the addressable market starts; faster ship; pure scope discipline.
7. **Subscription > usage-based pricing.** Subscriptions align our incentives with reduced cognitive load (not more agent activity).
8. **Personal product > team product at MVP.** B2B team plans are v2. Solo users first.

23.5.9 Post-launch retrospective

Year 1, looking back:

What worked:

- Trust progression model was right. Users who completed Phase 0-2 retained at 80%+. Users who skipped Phase 0 (impatient) churned at 60%.
- Audit log as primary surface was a hit. Users referenced it frequently.

- Triage accuracy reached 94% within 2 months — thanks to reasoning trace + correction loop.
- Word of mouth among founders/PMs. The product spoke to a specific power-user pain.

What didn't:

- Calendar coordination was harder than expected. Multi-party scheduling has subtle politics ("don't schedule with Sara before Tom because Tom should hear first"). Couldn't fully automate; required user oversight longer than planned.
- Drafting tone — quality varied dramatically by user. Highly individual. Required heavy personalization, slowed time to value.
- Some users wanted full autonomy from day one and were frustrated by slow progression. Lost some.
- Errors are public when they happen. One viral negative tweet about Karya sending an inappropriate email cost us 3 weeks of trust.
- B2B demand was higher than expected. Team plans launched in v1.2, earlier than planned.
- Travel feature shipped late and was used less than expected. Most power users had EAs already; Karya wasn't the marginal value here.

What we changed in v1.x:

- Added "express trust" mode for users who explicitly request faster autonomy with clear acceptance of risk.
- Major tone personalization investment in v1.2.
- Incident response runbook for any error that goes public.
- B2B team plans launched in v1.2.
- De-emphasized travel; redirected resources to email/calendar excellence.

What we'd do differently:

- More user segmentation upfront. Founders need different things than ICs at companies.
- Tone personalization is core, not nice-to-have. Build it from day one.
- Public error response process from the beginning. We were caught flat-footed.
- B2B GTM earlier. The team product is more valuable than solo.

23.5.10 What would Google look for here?

Agentic AI is the frontier of product thinking. Strong signals:

- **Trust-building as the central design pattern.** Most agent products fail here.
- **Autonomy progression UX.** Phase 0-4 is structurally smart.
- **Boundary system thoroughness.** People, topic, action, time, threshold boundaries.
- **Audit log as primary surface.** Counterintuitive design choice; right one.

- **Specific user fears mapped to specific defenses.** Discovery → architecture → safety layers.
- **Honest about quality variance and tone challenges.** Not over-promising AI.

Likely follow-ups:

- "What's your moat 18 months out as Microsoft, Google, OpenAI launch competing agents?" (Strategic competition.)
- "Walk me through how you'd handle a major public incident — Karya sends a bad email that goes viral." (Crisis management.)
- "How do you think about the line between agent that proposes vs agent that acts?" (Agentic philosophy.)
- "What's your view on B2B vs B2C as the primary path for personal AI agents?" (Strategic direction.)
- "How do you know if your trust progression is too fast or too slow for a given user?" (Personalization sophistication.)

A strong PM has thought through each. Agentic products are where 2026-2027 PM thinking is being forged. If you can speak fluently about Karya's design — even though it's hypothetical — you're operating at the edge of the discipline.

23.5.11 Closing — what these five journeys teach

Five different products. Five different domains. Five different sets of decisions. The patterns recur:

- **Discovery rigor wins.** All five started with deep user research before building.
- **Trade-offs are explicit.** Each journey named 8 specific trade-offs accepted, not avoided.
- **Metrics trees discipline thinking.** Each journey decomposed success into actionable inputs.
- **Kill criteria define commitment.** Each journey said: here's what would make us stop.
- **Retrospectives are honest.** Each journey named what didn't work, not just what did.
- **AI is deliberate.** Each journey has a clear "where AI sits, where it doesn't" architecture.

These are not the only patterns. They are necessary ones. PMs who can produce work at this depth — for any product, in any domain — operate at Senior PM level and above.

If you've absorbed these journeys, you have five complete case studies you can draw from in interviews. You have five sets of design decisions you can defend. You have five worked examples of how to think.

The next chapter, Part 24, is the interview gauntlet. Use these journeys as your raw material.

End of Part 23.

\newpage